

Python Set Objects

Python sets are **unordered, mutable, non-repeating** collections. Like the sets in mathematics, sets are represented by braces.

To create a set in Python, write some *(hashable)* Python objects between braces seperating them with `,`.

Some examples to the hashable objects are integers, strings, floating point numbers, `True`, `False`, `None` and tuples. But lists, sets and dictionaries are not hashable. So you can not nest sets.

In [9]:
```
s = {'hello', 'world', 1, None, 1,(1,2)}
```

Even if you write the same item multiple times in a set items can not repeat.

Sets are **un-ordered**, meaning the order of items are not preserved. Therefore unlike lists and tuples you can not access items by index.

You can create a set with an iterable using `set()` function.

In [10]:
```
set('Hello world!')
```

Out[10]:
```
{' ', '!', 'H', 'd', 'e', 'l', 'o', 'r', 'w'}
```

Or you can create a set with comprehension syntax.

In [11]:
```
{x**2 for x in range(10) if x & 1}
```

Out[11]:
```
{1, 9, 25, 49, 81}
```

In comprehension `if` part is optional.

In [12]:
```
{x**2 for x in range(10)}
```

Out[12]:
```
{0, 1, 4, 9, 16, 25, 36, 49, 64, 81}
```

Empty Set

Warning: The empty set is created by `set()` **not** `{}`

Testing The Membership

With `in` keyword you can test if an object is in a set or not.

In [13]:
```
(1,2) in s
```

Out[13]:
```
True
```

Adding New Items

You can add new hashable items to a set. If it is already in the set then it is ignored.

In [14]:
```
s.add(3.14)
s
```

Out[14]:
```
{(1, 2), 1, 3.14, None, 'hello', 'world'}
```

You can also use `.update(iterable)` method to add multiple items using a set `iterable` (or any iterable).

In [15]:
```
s.update({1,2,3})
s
```

Out[15]:
```
{(1, 2), 1, 2, 3, 3.14, None, 'hello', 'world'}
```

Removing An Item From The Set

`.discard(obj)` and `.remove(obj)` method are both used for removing an object `obj` from the set. The difference is `.remove(obj)` yields `ValueError` if `obj` is not in the set. While `.discard(obj)` simply ignores if it does not exist.

In [16]:
```
s.remove(3.14)
s.discard(3.14)
s
```

Out[16]:
```
{(1, 2), 1, 2, 3, None, 'hello', 'world'}
```

Matematical Operations On Sets

You can do mathematical set operations on sets with:

- `.union(iterable)`
- `.intersection(iterable)`
- `.difference(iterable)`
- `.symmetric_difference(iterable)`
- `.issubset(iterable)`
- `.issuperset(iterable)`
- `.isdisjoint(iterable)`

methods. Where `iterable` is a set, list, tuple or any iterable.

Warning: These methods do not modify the set itself. But they create a new set.

In [17]:
```
t = {'a','b','c'}
```

In [18]:
```
s.union(t)
```

Out[18]:
```
{(1, 2), 1, 2, 3, None, 'a', 'b', 'c', 'hello', 'world'}
```

In [19]:
```
s.intersection(t)
```

Out[19]:
```
set()
```

In [20]:
```
s.difference(t)
```

Out[20]:
```
{(1, 2), 1, 2, 3, None, 'hello', 'world'}
```

In [21]:
```
s.symmetric_difference(t)
```

Out[21]:
```
{(1, 2), 1, 2, 3, None, 'a', 'b', 'c', 'hello', 'world'}
```

In [22]:
```
s.issubset(t)
```

Out[22]:
```
False
```

In [23]:
```
s.issuperset(t)
```

Out[23]:
```
False
```

In [24]:
```
s.isdisjoint(t)
```

Out[24]:
```
True
```